

Javascript

JavaScript is a high-level, versatile programming language that serves as one of the core technologies of the World Wide Web, alongside HTML and CSS

It was created by Brendan Eich at Netscape in 1995 to add interactivity and dynamic behavior to web pages, such as image carousels, interactive maps, and animated graphics.

Ways to add JavaScript in HTML

There are **3 main ways to add JavaScript in HTML**. Each has its own use case:

- **Inline JavaScript :** You write JavaScript directly inside an HTML tag.

Example: `<button onclick="alert('Hello!')">Click Me</button>`

- **Internal JavaScript (Embedded) :** You write JavaScript inside the `<script>` tag within the HTML file.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JS</title>
</head>
<body>

<script>
  function greet() {
    alert("Hello from internal JS!");
  }
</script>

<button onclick="greet()">Click Me</button>

</body>
</html>
```

- **External JavaScript :** You write JavaScript in a separate .js file and link it to HTML.

HTML file:

```
<script src="script.js"></script>
```

JavaScript file (script.js):

```
function greet() {  
    alert("Hello from external JS!");  
}
```

JavaScript Comments : JavaScript comments can be used to explain JavaScript code, and to make it more readable.

- **Single Line Comments :** Single line comments start with `//`. Any text between `//` and the end of the line will be ignored by JavaScript (will not be executed)

```
let x = 5;    // Declare x, give it the value of 5
```

- **Multi-line Comments :** Multi-line comments start with `/*` and end with `*/`. Any text between `/*` and `*/` will be ignored by JavaScript

```
/*  
document.getElementById("myH").innerHTML = "My First Page";  
document.getElementById("myP").innerHTML = "My first paragraph.";  
*/
```

JavaScript Variables : JavaScript variables are containers for data.

JavaScript variables can be **declared** in 4 ways:

- Modern JavaScript
 - Using let
 - let x = 5;
 - let z = x + y;
 - Using const
 - const x = 5;
 - const z = x + y;
- Older JavaScript
 - Using var
 - var x = 5;
 - var z = x + y;
 - Automatically (Not Recommended)

JavaScript Let

- The let keyword was introduced in ES6 (2015)
- Variables declared with let have **Block Scope**

- Variables declared with let must be **Declared** before use
- Variables declared with let cannot be **Redeclared** in the same scope

JavaScript Const

- The const keyword was introduced in ES6 (2015)
- Variables defined with const cannot be **Redeclared**
- Variables defined with const cannot be **Reassigned**
- Variables defined with const have **Block Scope**

JavaScript Datatypes : JavaScript has 8 Datatypes

Type	Description
String	A text of characters enclosed in quotes
Number	A number representing a mathematical value
Bigint	A number representing a large integer
Boolean	A data type representing true or false
Object	A collection of key-value pairs of data
Undefined	A primitive variable with no assigned value
Null	A primitive value representing object absence
Symbol	A unique and primitive identifier

// String

```
let color = "Yellow";
let lastName = "Johnson";
```

// Number

```
let length = 16;
let weight = 7.5;
```

// BigInt

```
let x = 1234567890123456789012345n;
let y = BigInt(1234567890123456789012345)
```

// Boolean

```
let x = true;
let y = false;
```

// Object

```
const person = {firstName:"John", lastName:"Doe"};
```

// Array object

```
const cars = ["Saab", "Volvo", "BMW"];
```

// Date object

```
const date = new Date("2022-03-25");
```

// Undefined

```
let x;
```

```
let y;
```

// Null

```
let x = null;
```

```
let y = null;
```

// Symbol

```
const x = Symbol();
```

```
const y = Symbol();
```

typeof Operator : You can use the JavaScript typeof operator to find the type of a JavaScript variable.

```
typeof 314      // Returns "number"
```

```
typeof "John Doe" // Returns "string"
```

Types of JavaScript Operators :

- **Arithmetic Operators** : are used to perform arithmetic on numbers

- let a = 3;

```
let x = (100 + 50) * a;
```

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

- **Assignment Operators** : Assignment operators assign values to JavaScript variables.

- let x = 10;
x += 5;

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

- **Comparison Operators** : Comparison operators are used to **compare two values**.

- let x = 5;
let result = x > 8;

Operator	Description	Example
==	equal to	x == 5
===	equal value and equal type	x === 5
!=	not equal	x != 5
!==	not equal value or not equal type	x !== 5
>	greater than	x > 5
<	less than	x < 5
>=	greater than or equal to	x >= 5
<=	less than or equal to	x <= 5

- **Logical Operators** : are used to determine the logic between variables or values

- (x < 10 && y > 1) is true
- (x === 5 || y === 5) is false

Operator	Description
&&	logical and
	logical or
!	logical not

JavaScript Conditionals

Conditional Statements allow us to perform **different actions** for **different conditions**. Conditional statements **run different code** depending on **true** or **false** conditions.

- **If** : Use if to specify a **code block** to be executed, if a specified condition is true
- **If...else** : Use else to specify a **code block** to be executed, if the same condition is false
- **if...else if...else** : Use else if to specify a **new condition** to test, if the first condition is false
- **Switch** : Use switch to specify many **alternative code blocks** to be executed

- **ternary (?:) :** Use (?:) (ternary) as a **shorthand** for if...else

The if...else if...else

```
if (condition1) {
    // code to execute if condition1 is true
} else if (condition2) {
    // code to execute if the condition1 is false and condition2 is true
} else {
    // code to execute if the condition1 is false and condition2 is false
}
```

The switch Statement

```
switch(expression) {
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
}
```

Ternary Operator (?:)

condition ? expression1 : expression2

JavaScript Loops : Loops can execute a block of code a number of times.

- **The For Loop :** The for statement creates a loop with 3 optional expressions
 - for (*expr1*; *expr2*; *expr*) {

// code block to be executed

 }
 - for (let *i* = 0; *i* < 5; *i*++) {

 text += "The number is " + *i* + "
";

 }
- **The While Loop :** The while loop loops through a block of code as long as a specified condition is true.
 - while (*condition*) {

// code block to be executed

 }

- while (i < 10) {
 text += "The number is " + i;
 i++;
}

- **The Do While Loop** : The do while loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

- do {
 // code block to be executed
}
 while (condition);
- do {
 text += "The number is " + i;
 i++;
}
 while (i < 10);

The Break Statement

- The break statement "jumps out" of loops and switches.
- The break statement **terminates** the execution of a **loop** or a **switch** statement
 - for (let i = 0; i < 10; i++) {
 if (i === 3) { break; }
 text += "The number is " + i + "
";
}
 - The break statement exits a loop or block and transfers the control to the labeled statement.
 - let text = "";

```
loop1: for (let j = 1; j < 5; j++) {  
  loop2: for (let i = 1; i < 5; i++) {  
    if (i === 3) { break loop1; }  
    text += i;  
  }  
}
```

The Continue Statement

- The continue statement **skips the current iteration** in a loop.
- The remaining code in the iteration is skipped and processing moves to the next iteration.
 - for (let i = 1; i < 10; i++) {
 if (i === 3) { continue; }
 text += "The number is " + i + "
";
}

JavaScript Functions

- Functions are **reusable code blocks** designed to perform a particular task.
- Functions **are executed** when they are **called** or **invoked**.
- Functions are **fundamental** in all programming languages.

```
function sayHello() {  
    return "Hello World";  
}  
SayHello();
```

- The code inside a function is **NOT executed** when the function is **defined**.
- The code inside a function **will execute** when "something" **invokes** the function:
 - When it is called from JavaScript code
 - When an event occurs (a user clicks a button)
 - Automatically (self invoked)

JavaScript Function Expressions : A function expression is a **function stored in a variable**.

```
// Standard Function  
function multiply(a, b) {  
    return a * b;  
}  
  
// Function Expression  
const multiply = function(a, b) {  
    return a * b;  
};
```

JavaScript Arrow Functions

- **Arrow Functions** allow a shorter syntax for **function expressions**.
- You can skip the **function** keyword, the **return** keyword, and the **curly brackets**:
 - `const multiply = (a, b) => a * b;`
- An arrow function uses the **=>** symbol.
- An arrow function is always written as a function expression.